INTELIGENCIA DE NEGOCIOS · GRUPO 02

Los ETL de los Hechos del Data Mart DM_Cafeteria

Segunda guía de estudio: cómo se cargan FactVenta, FactRecepcionProducto, FactDelivery y FactPago, con cada componente SSIS y cada consulta SQL explicada.

Fuentes analizadas

· `Package.dtsx` del proyecto `ETL_Cafeteria_Grupo02` (Visual Studio 2026) — "Contenedor Hechos" y tareas de control
· `DM_Cafeteria.sql` — definición de las 4 tablas de hechos, `ETL_Control` y sus procedimientos
· `Cafeteria.sql` — tablas origen, triggers y generador de datos

Requisito previo: haber leído la guía de dimensiones. Aquí se asume que ya sabes qué es una clave sustituta, el componente Búsqueda (Lookup) y la carga rápida.

Contenido

1. **Parte 1 · Lo general**
    1. ¿Qué es una tabla de hechos? Grano, medidas y tipos de hecho
    2. Los 4 hechos de DM_Cafeteria de un vistazo
    3. ¿Cuál es el propósito de un ETL de hechos?
    4. Dónde están los hechos dentro del paquete
2. **Parte 2 · El patrón que comparten los 4 hechos**
    1. Los componentes del flujo de datos
    2. Propiedades internas de cada componente
    3. Las 3 formas de tratar un "no encontrado"
    4. Medidas calculadas: por qué no se cargan desde SSIS
    5. Recorrido de una fila, paso a paso
3. **Parte 3 · Cada hecho en detalle**
    1. FactVenta
    2. FactRecepcionProducto
    3. FactDelivery (snapshot acumulativo: inserta y actualiza)
    4. FactPago
4. **Parte 4 · El control de la ejecución**
    1. ETL_Control y la marca de inicio
    2. Calcular Filas Procesadas
    3. Fin OK y Fin Error
    4. ¿Qué pasa al ejecutar el paquete otra vez?
5. **Parte 5 · Para la defensa**
    1. Cuadro comparativo de los 4 hechos
    2. Consultas de conciliación (demostrar que cuadra)
    3. Riesgos y limitaciones honestas
    4. Banco de preguntas probables del docente
    5. Glosario

> **Misma convención que la guía anterior.** Verde = **"En simple"**; marrón = **pregunta probable del docente**; rojo = **punto delicado**; azul = dato técnico útil.

Parte 1 · Lo general

1.1 ¿Qué es una tabla de hechos?

> **En simple.** Las dimensiones son los **sustantivos** (producto, cliente, sucursal, día). Los hechos son los **verbos con números**: "el cliente X compró 2 lattes por S/ 25", "el proveedor Y entregó 40 unidades a S/ 4 cada una", "el repartidor Z tardó 38 minutos". Cada fila de un hecho es **un evento** que ocurrió en el negocio.

Una fila de hecho tiene siempre tres tipos de columnas:

Tipo de columna	Qué es	Ejemplo en FactVenta
Claves de dimensión (FK)	Enteros que apuntan a cada dimensión: dicen *cuándo, qué, quién, dónde*.	`TiempoKey, ProductoKey, ClienteKey, SucursalKey, EmpleadoKey, TipoPedidoKey`
Medidas	Los números que se suman, promedian o cuentan.	`Cantidad, PrecioUnitario, DescuentoImporte, VentaNeta, MargenEstimado…`
Dimensiones degeneradas y auditoría	Identificadores del origen que no merecen tabla propia (número de pedido, línea) y columnas técnicas de la carga.	`PedidoIDOrigen, LineaOrigen, HoraPedido, IdControlCarga, FechaCargaUTC`

El grano (granularidad)

El **grano** responde a la pregunta: *"¿qué representa exactamente una fila?"*. Es la decisión más importante de un hecho, porque define qué preguntas se pueden responder y cómo se evitan duplicados. En el DM cada hecho tiene su grano declarado en los comentarios de `DM_Cafeteria.sql` y protegido con una restricción `UNIQUE`:

Hecho	Grano: una fila por…	Restricción que lo garantiza
FactVenta	línea de pedido completado (`PedidoID` + `Linea`)	`UQ_FactVenta_Origen UNIQUE(PedidoIDOrigen, LineaOrigen)`
FactRecepcionProducto	línea de ingreso de mercadería (`IngresoID` + `Linea`)	`UQ_Recepcion_Origen UNIQUE(IngresoIDOrigen, LineaOrigen)`
FactDelivery	pedido con delivery (`PedidoID`)	`UQ_FactDelivery_Origen UNIQUE(PedidoIDOrigen)`
FactPago	cobro (`PagoID`); un pedido puede tener 2 pagos (pago mixto)	`UQ_FactPago_Origen UNIQUE(PagoIDOrigen)`

Tipos de tabla de hechos (Kimball)

Tipo	Cómo se comporta	En el proyecto
Transaccional	Una fila por evento puntual. Una vez cargada **no cambia**: solo se insertan filas nuevas.	FactVenta, FactRecepcionProducto, FactPago
Snapshot acumulativo	Una fila por **proceso** con varios hitos (pedido → salida → entrega). La fila se crea al empezar el proceso y se **actualiza** a medida que ocurren los hitos.	FactDelivery
Snapshot periódico	Una foto cada cierto tiempo (ej. stock al cierre de cada día).	No se usa

Aditividad de las medidas

- **Aditivas**: se pueden sumar por cualquier dimensión. Ej.: `Cantidad`, `VentaNeta`, `ImporteRecibido`, `MontoCobrado`.
- **No aditivas**: sumarlas no tiene sentido. Ej.: `PrecioUnitario`, `CostoUnitario` (se promedian o se recalculan como importe/cantidad).

- **Para promediar**: `MinutosPreparacion`, `MinutosTransporte`, `MinutosTotales` de FactDelivery se suman técnicamente, pero lo útil es el promedio.

1.2 Los 4 hechos de DM_Cafeteria de un vistazo

Hecho	Proceso de negocio	Tablas origen	Filtro	Dimensiones	Medidas principales
FactVenta	Venta de productos	PedidoVenta + DetallePedidoVenta	Solo pedidos **Completados**	Tiempo, Producto, Cliente, Sucursal, Empleado, TipoPedido	Cantidad, Precio, Descuento, Costo, ImporteBruto, VentaNeta, Margen
FactRecepcionProducto	Compra/recepción a proveedores	IngresoProducto + DetalleIngresoProducto	Ninguno	Tiempo, Sucursal, Empleado, Producto, Proveedor	Cantidad, CostoUnitario, ImporteRecibido
FactDelivery	Despacho y entrega a domicilio	Delivery + PedidoVenta	Ninguno (incluye pendientes)	Tiempo ×3, Sucursal, Cliente, Empleado (repartidor), Vehículo	CargoEnvio, Entregada, Minutos de preparación/transporte/totales
FactPago	Cobro	Pago + ComprobantePago + PedidoVenta	Solo pedidos **Completados**	Tiempo, Sucursal, Cliente, TipoPedido, MetodoPago	MontoCobrado

1.3 ¿Cuál es el propósito de un ETL de hechos?

> **En simple.** El ETL de un hecho toma los eventos del sistema de la cafetería, les cambia los "códigos del sistema" (ProductoID 16) por los "códigos del Data Mart" (ProductoKey 16 o el que sea), les pone un sello con el número de ejecución y los guarda, **sin repetir** los que ya estaban.

Concretamente, cada ETL de hecho hace 6 cosas:

1. **Extraer** los eventos uniendo cabecera y detalle (pedido + líneas, ingreso + líneas...).
2. **Filtrar** solo lo que debe medirse (ventas y pagos: solo pedidos completados).
3. **Sustituir claves**: cada ID del origen (`ProductoID`, `ClienteID`, la fecha...) se reemplaza por la **clave sustituta** de su dimensión mediante componentes Búsqueda. Este es el corazón del ETL de hechos.
4. **Sellar la carga**: añadir `IdControlCarga` (el número de ejecución de `ETL_Control`) para saber qué ejecución cargó cada fila.
5. **Evitar duplicados**: comprobar contra el propio hecho, por su grano, si la fila ya fue cargada antes.
6. **Cargar**: insertar lo nuevo (y en FactDelivery, además, actualizar lo que avanzó de hito).

Diferencias con los ETL de dimensiones

Aspecto	ETL de dimensión	ETL de hecho
Qué carga	Catálogos (pocas filas)	Eventos (decenas de miles de filas)
Búsquedas	1 Lookup contra su propia tabla	Varios Lookups contra **las dimensiones** + 1 contra el propio hecho
¿Actualiza?	Sí, siempre (SCD 1)	No (Venta, Recepción, Pago) · Sí (Delivery)
Auditoría	Sin columnas de control	`IdControlCarga` + `FechaCargaUTC`
Cálculos	CASE / CONCAT en el SELECT	Columnas calculadas PERSISTED en la tabla del DM

1.4 Dónde están los hechos dentro del paquete

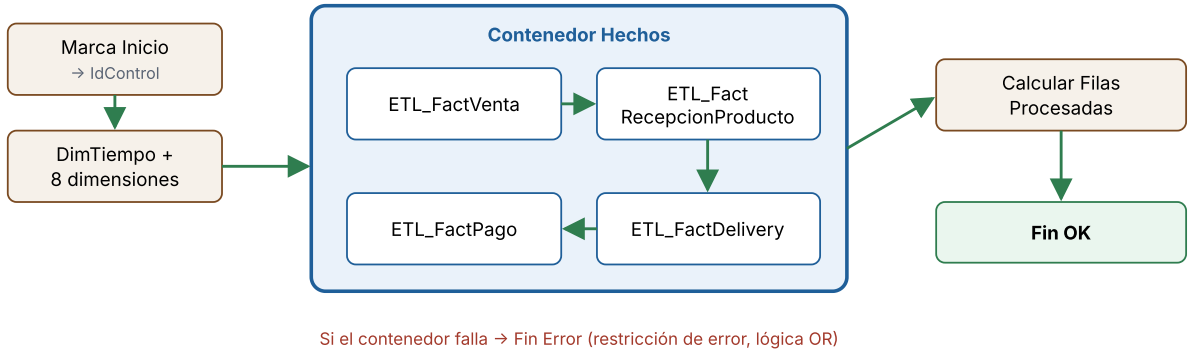


Figura 1. El Contenedor Hechos: 4 tareas Flujo de datos en cadena (Venta → Recepción → Delivery → Pago).

- Los 4 flujos están en un **contenedor de secuencias** llamado `Contenedor Hechos`, unidos por restricciones de **éxito**: FactVenta → FactRecepcionProducto → FactDelivery → FactPago.
- El contenedor solo empieza cuando terminó bien el `Contenedor de dimensiones` (que a su vez espera a `Poblar DimTiempo`).
- Antes de todo, `Marca Inicio` dejó en la variable `User::IdControl` el número de esta ejecución; los 4 hechos lo graban en cada fila.
- Al terminar, `Calcular Filas Procesadas` cuenta cuántas filas de hechos cargó esta ejecución y `Fin OK` cierra el registro.

> **Pregunta probable:** "¿Importa el orden entre los hechos?"
> **Respuesta:** No hay dependencias entre hechos (ningún hecho tiene FK a otro), así que el orden es una decisión de organización: primero el hecho principal (ventas), luego compras, entregas y cobros. Ir en cadena permite saber exactamente cuál falló y evita que dos cargas masivas compitan por recursos.

Parte 2 · El patrón que comparten los 4 hechos

En simple. Los 4 flujos siguen la misma receta: **"leo los eventos → traduzco cada código del sistema a la clave del Data Mart (una búsqueda por dimensión) → le pego el número de ejecución → pregunto si ese evento ya lo cargué → si es nuevo lo inserto"**. FactDelivery añade un paso más: si ya lo tenía pero cambió, lo actualiza.

2.1 Los componentes del flujo de datos

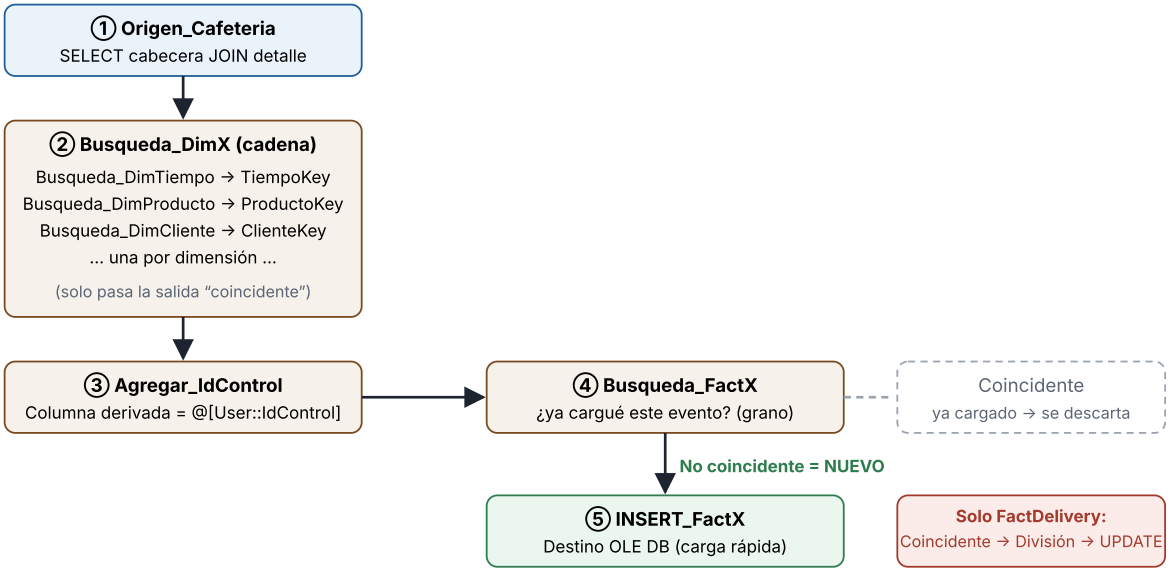


Figura 2. Plantilla común de los flujos de hechos.

#	Componente	Función
①	Origen de OLE DB `Origen_Cafeteria`	**Extrae** los eventos con un `SELECT` que une cabecera y detalle, convierte fechas (`CONVERT(date, …)`) para poder buscarlas en DimTiempo y renombra los IDs con sufijo `IDOrigen`.
②	Búsquedas de dimensión `Busqueda_DimX`	**Sustituyen claves**. Cada búsqueda recibe un ID del origen (ej. `ClienteIDOrigen`) y devuelve la clave sustituta (`ClienteKey`). Están en **cadena**: la salida coincidente de una entra a la siguiente.
③	Columna derivada `Agregar_IdControl`	**Sella la carga**: agrega una columna nueva con el valor de la variable `User::IdControl`.
④	Búsqueda sobre el hecho `Busqueda_FactX`	**Evita duplicados**: busca la fila en la propia tabla de hechos usando la clave del grano (ej. `PedidoIDOrigen + LineaOrigen`). Si ya existe, no se vuelve a cargar.
⑤	Destino de OLE DB `INSERT_FactX`	**Carga** las filas nuevas con carga rápida.

2.2 Propiedades internas de cada componente

① Origen de OLE DB

Propiedad	Valor	Significado
AccessMode	`2` = Comando SQL	Se escribe una consulta con JOIN y WHERE.
Conexión	`localhost.Cafeteria`	Lee del sistema transaccional.
Tipos de salida	`DT_DBDATE`, `DT_DBTIME2`, `DT_DBTIMESTAMP2`, `DT_NUMERIC(12,2)`, `DT_I2`, `DT_I4`, `DT_STR`	date, time(0), datetime2(0), decimal(12,2), smallint, int, varchar. Coinciden con los tipos del DM, por eso **no hace falta** una transformación "Conversión de datos".

② Búsquedas de dimensión (Busqueda_DimX)

Propiedad	Valor	Significado
Consulta de referencia	`SELECT XKey, XIDOrigen FROM dbo.DimX;`	Solo trae **2 columnas**: la clave natural para unir y la sustituta para devolver. Así la caché en memoria es mínima (a diferencia de los Lookups de dimensión, aquí no se necesitan los atributos).
Para DimTiempo	`SELECT TiempoKey, Fecha FROM dbo.DimTiempo;`	Se une por **fecha** (columna `date`), no por un ID. Por eso el origen convierte `FechaHora` a `date`.
CacheType	`0` = Caché completa	Cada dimensión se carga entera en memoria antes de procesar filas.
Conexión	`localhost.DM_Cafeteria`	Las claves se buscan en el Data Mart.
NoMatchBehavior	Varía (ver 2.3)	Qué hacer si el ID no existe en la dimensión.
Columna agregada	`XKey` (`DT_I4`)	La clave sustituta queda como columna nueva en el flujo y luego se mapea al INSERT.

③ Columna derivada (Agregar_IdControl)

Propiedad	Valor / significado
Nombre de columna	`IdControlCarga` (en FactDelivery se llama `IdControlEjecucion`)
Expresión	`@[User::IdControl]` — lee la variable del paquete (en expresiones SSIS las variables se escriben `@[Espacio::Nombre]`).
Acción	"Agregar como nueva columna"
Tipo	`DT_I8` (entero de 8 bytes) = `bigint`, igual que `IdControlCarga` en el DM y que la variable (tipo Int64).

> **¿Para qué sirve IdControlCarga?** Es una FK a `ETL_Control(IdControl)`. Permite responder "¿qué ejecución cargó esta fila?", contar lo cargado en una ejecución (lo hace *Calcular Filas Procesadas*) y, si una carga saliera mal, identificar y borrar exactamente sus filas: `DELETE FROM FactVenta WHERE IdControlCarga = 7`.

④ Búsqueda sobre el propio hecho (Busqueda_FactX)

Propiedad	Valor / significado
Consulta	Ej. `SELECT VentaKey, PedidoIDOrigen, LineaOrigen FROM dbo.FactVenta;`
Columnas de unión	La clave del **grano**. Puede ser **compuesta** (dos columnas): `PedidoIDOrigen` + `LineaOrigen`.
NoMatchBehavior	`1` = redirigir: lo que **no** existe sale por "No coincidente" → INSERT.
Salida coincidente	No conectada (Venta, Recepción, Pago): las filas ya cargadas se descartan. En Delivery sí está conectada (va a la División condicional).

> **En simple.** Esto convierte al ETL en una **carga incremental por clave**: se leen todos los eventos del origen, pero solo se insertan los que el DM aún no tiene. Por eso el paquete se puede ejecutar todos los días sin duplicar ventas.

⑤ Destino de OLE DB (INSERT_FactX)

Propiedad	Valor	Significado
AccessMode	`3` = carga rápida	Bulk insert.
FastLoadKeepIdentity	`false`	`VentaKey`, `RecepcionKey` … (`bigint IDENTITY`) las genera SQL Server.
FastLoadOptions	`TABLOCK, CHECK_CONSTRAINTS`	Bloqueo de tabla para velocidad y validación de las restricciones CHECK y FOREIGN KEY durante la carga masiva.
Columnas no mapeadas	Key, calculadas, `FechaCargaUTC`	La Key es IDENTITY; las medidas calculadas las llena SQL Server; `FechaCargaUTC` tiene `DEFAULT SYSUTCDATETIME()`.
Error de entrada	`FailComponent`	Una fila que viole una restricción hace fallar el flujo → Fin Error.

2.3 Las 3 formas de tratar un "no encontrado"

Este es un tema fino que distingue a quien realmente revisó el paquete. En el XML, cada Búsqueda de hecho tiene una de estas tres configuraciones:

Configuración	Propiedades	Qué pasa si no encuentra la clave	Dónde se usa
A. Fallar	`NoMatchBehavior = 0` (tratar como error) + salida con `FailComponent`	El componente **falla** y con él todo el paquete → Fin Error.	`Busqueda_DimTiempo` de FactVenta
B. Ignorar el error	`NoMatchBehavior = 0` + salida con `IgnoreFailure`	La fila **continúa** por la salida coincidente, pero la columna buscada queda en **NULL**.	`Busqueda_DimTiempoEntrega` de FactDelivery
C. Redirigir	`NoMatchBehavior = 1` (redirigir a salida no coincidente)	La fila sale por "No coincidente". Si esa salida **no está conectada**, la fila se **descarta en silencio**.	Todas las demás búsquedas de dimensión de los 4 hechos

> **Por qué B es una decisión inteligente.** Un delivery que salió pero **aún no se entrega** tiene `FechaEntrega = NULL`. Buscar NULL en DimTiempo nunca coincide. Con "Ignorar error", la fila sigue y `TiempoEntregaKey` queda en NULL, que es exactamente lo que exige la tabla: `CK_FactDelivery_Hito` pide que `FechaEntrega` y `TiempoEntregaKey` sean ambos NULL o ambos con valor.

Punto delicado de la opción C. En las búsquedas de dimensión con opción C la salida "No coincidente" no está conectada a nada. Si algún día llegara un evento cuyo producto, cliente, etc. no está en la dimensión, esa fila **se perdería sin aviso** (el paquete terminaría en OK). Hoy no ocurre porque las dimensiones se cargan justo antes desde el mismo origen y las FK del origen garantizan que todo ID existe. Si el docente lo pregunta, la mejora es conectar esa salida a un **Recuento de filas** o a una tabla de rechazos, o cambiarla a "Fallar" (opción A). Ver Parte 5.3.

2.4 Medidas calculadas: por qué no se cargan desde SSIS

En `DM_Cafeteria.sql` varias medidas son **columnas calculadas PERSISTED**. El ETL solo envía los valores base y SQL Server calcula el resto al insertar:

Hecho	Columna calculada	Fórmula
FactVenta	ImporteBruto	`Cantidad * PrecioUnitario`
	VentaNeta	`Cantidad * PrecioUnitario - DescuentoImporte`
	CostoTotalEstimado	`Cantidad * CostoUnitarioEstimado`
	MargenEstimado	`Cantidad * (PrecioUnitario - CostoUnitarioEstimado) - DescuentoImporte`
FactRecepcionProducto	ImporteRecibido	`Cantidad * CostoUnitario` como `decimal(22,2)`
FactDelivery	Entregada	`CASE WHEN FechaEntrega IS NULL THEN 0 ELSE 1 END`
	MinutosPreparacion	`DATEDIFF_BIG(SECOND, FechaHoraPedido, FechaSalida) / 60.0`
	MinutosTransporte	`DATEDIFF_BIG(SECOND, FechaSalida, FechaEntrega) / 60.0`
	MinutosTotales	`DATEDIFF_BIG(SECOND, FechaHoraPedido, FechaEntrega) / 60.0`

Ventajas: la fórmula vive en un solo lugar (no se puede "olvidar" en el ETL), siempre es consistente con los valores base, y al ser `PERSISTED` se guarda físicamente y se puede indexar (ej. `VentaNeta` está en el `INCLUDE` de `IX_FactVenta_TiempoSucursal`). En FactDelivery, si `FechaEntrega` es NULL, `DATEDIFF_BIG` devuelve NULL: un delivery pendiente no aparece con "0 minutos".

Pregunta probable: "¿Por qué dividir segundos entre 60.0 y no usar DATEDIFF(MINUTE...)?"
Respuesta: `DATEDIFF(MINUTE…)` cuenta cuántas veces se cruza el límite de minuto, no la duración real (de 10:00:59 a 10:01:00 da 1 minuto). Calcular en segundos y dividir entre `60.0` (decimal) da la duración exacta con decimales.

2.5 Recorrido de una fila, paso a paso

Ejemplo: línea 1 de un pedido completado de la sucursal 2, del 15/12/2025 a las 19:40, cliente 350, mozo 17, "En Mesa", 2 lattes a S/ 12.50.

Componente	Qué le pasa a la fila
① Origen	Sale con `PedidoIDOrigen`, `LineaOrigen=1`, `FechaPedido=2025-12-15`, `HoraPedido=19:40:00`, `ProductoIDOrigen=19`, `ClienteIDOrigen=350`, `SucursalIDOrigen=2`, `EmpleadoIDOrigen=17`, `TipoPedidoIDOrigen=1`, `Cantidad=2`, precio, descuento y costo.
② Busqueda_DimTiempo	Busca 2025-12-15 en DimTiempo → agrega `TiempoKey`.
② Busqueda_DimProducto … DimTipoPedido	Cada búsqueda agrega su Key: `ProductoKey`, `ClienteKey`, `SucursalKey`, `EmpleadoKey`, `TipoPedidoKey`.
③ Agregar_IdControl	Agrega `IdControlCarga = 5` (si es la 5.ª ejecución).
④ Busqueda_FactVenta	Busca (PedidoIDOrigen, 1) en FactVenta. **1.ª ejecución**: no existe → "No coincidente". **Ejecuciones siguientes**: existe → "Coincidente" → se descarta.
⑤ INSERT_FactVenta	Se inserta. SQL Server asigna `VentaKey`, pone `FechaCargaUTC` y calcula `ImporteBruto = 25.00`, `VentaNeta`, `CostoTotalEstimado` y `MargenEstimado`.

Parte 3 · Cada hecho en detalle

3.1 FactVenta — ETL_FactVenta

> **En simple.** Es el hecho principal: **cada producto vendido en cada pedido completado**, con cuánto se vendió, a qué precio, con qué descuento y cuánto margen dejó. Responde preguntas como "¿qué producto deja más margen en la sucursal California los fines de semana?".

Grano: una fila por línea de pedido completado. **Tipo:** transaccional (solo inserta). **Volumen esperado en la 1.ª carga:** ≈ 105 905 filas (≈ 35 310 pedidos completados × 1 a 5 líneas).

① Origen_Cafeteria — consulta de extracción

```
SELECT
    P.PedidoID AS PedidoIDOrigen,
    D.Linea    AS LineaOrigen,

    CONVERT(date, P.FechaHora)    AS FechaPedido,
    CONVERT(time(0), P.FechaHora) AS HoraPedido,

    D.ProductoID   AS ProductoIDOrigen,
    P.ClienteID    AS ClienteIDOrigen,
    P.SucursalID   AS SucursalIDOrigen,
    P.EmpleadoID   AS EmpleadoIDOrigen,
    P.TipoPedidoID AS TipoPedidoIDOrigen,

    D.Cantidad,
    D.PrecioUnitario,
    D.Descuento AS DescuentoImporte,
    D.CostoUnitarioEstimado
FROM dbo.PedidoVenta AS P
INNER JOIN dbo.DetallePedidoVenta AS D
    ON P.PedidoID = D.PedidoID
WHERE P.EstadoPedidoID = 2;
```

Parte	Explicación
`FROM PedidoVenta P INNER JOIN DetallePedidoVenta D`	La **cabecera** (P) tiene fecha, sucursal, cliente, empleado y canal; el **detalle** (D) tiene producto, cantidad y precios. El JOIN "baja" los datos de la cabecera a cada línea → el resultado ya está al grano de línea. INNER porque un pedido completado siempre tiene detalle (el trigger `TR_Pedido_Integridad` rechaza cerrar un pedido sin líneas).
`WHERE P.EstadoPedidoID = 2`	Solo pedidos **Completados**. Se excluyen los **Abiertos** (1), que todavía pueden cambiar, y los **Cancelados** (3), que no son ventas. Por eso el DM no necesita una dimensión de estado.
`CONVERT(date, P.FechaHora) AS FechaPedido`	Solo el día: es la columna que se busca en DimTiempo (que tiene grano diario).
`CONVERT(time(0), P.FechaHora) AS HoraPedido`	Solo la hora, sin segundos fraccionarios. Se guarda **en el hecho** (no hay DimHora) y permite analizar ventas por franja horaria.
`D.Descuento AS DescuentoImporte`	Renombrado para dejar claro que es un **importe** (soles), no un porcentaje.
`D.PrecioUnitario` (no `Producto.PrecioActual`)	Se toma el precio **al que realmente se vendió**, guardado en la línea. El precio actual del catálogo puede haber cambiado después.
Sufijo `IDOrigen`	Indica que todavía son IDs del sistema; las búsquedas los reemplazarán por Keys.

② Cadena de búsquedas de dimensión

Orden	Componente	Entra	Une con	Devuelve	Si no encuentra
1	Busqueda_DimTiempo	FechaPedido	DimTiempo.Fecha	TiempoKey	**Falla** (A)
2	Busqueda_DimProducto	ProductoIDOrigen	DimProducto.ProductoIDOrigen	ProductoKey	Redirige (C)
3	Busqueda_DimCliente	ClienteIDOrigen	DimCliente.ClienteIDOrigen	ClienteKey	Redirige (C)
4	Busqueda_DimSucursal	SucursalIDOrigen	DimSucursal.SucursalIDOrigen	SucursalKey	Redirige (C)
5	Busqueda_DimEmpleado	EmpleadoIDOrigen	DimEmpleado.EmpleadoIDOrigen	EmpleadoKey	Redirige (C)
6	Busqueda_DimTipoPedido	TipoPedidoIDOrigen	DimTipoPedido.TipoPedidoIDOrigen	TipoPedidoKey	Redirige (C)

Consultas de referencia (todas con la misma forma):

```
SELECT TiempoKey, Fecha FROM dbo.DimTiempo;
SELECT ProductoKey, ProductoIDOrigen FROM dbo.DimProducto;
SELECT ClienteKey, ClienteIDOrigen FROM dbo.DimCliente;
SELECT SucursalKey, SucursalIDOrigen FROM dbo.DimSucursal;
SELECT EmpleadoKey, EmpleadoIDOrigen FROM dbo.DimEmpleado;
SELECT TipoPedidoKey, TipoPedidoIDOrigen FROM dbo.DimTipoPedido;
```

③ Agregar_IdControl y ④ Busqueda_FactVenta

```
-- ③ Columna derivada
IdControlCarga = @[User::IdControl]          -- DT_I8

-- ④ Consulta de referencia; une por PedidoIDOrigen + LineaOrigen
SELECT VentaKey, PedidoIDOrigen, LineaOrigen FROM dbo.FactVenta;
```

La búsqueda sobre el hecho usa una **clave compuesta**: un mismo pedido tiene varias líneas, así que el pedido solo no basta. Solo la salida **No coincidente** está conectada (→ INSERT).

⑤ INSERT_FactVenta — mapeo de columnas

Columna del flujo	Columna de FactVenta	Columna del flujo	Columna de FactVenta
TiempoKey	TiempoKey	PedidoIDOrigen	PedidoIDOrigen
ProductoKey	ProductoKey	LineaOrigen	LineaOrigen
ClienteKey	ClienteKey	HoraPedido	HoraPedido
SucursalKey	SucursalKey	Cantidad	Cantidad
EmpleadoKey	EmpleadoKey	PrecioUnitario	PrecioUnitario
TipoPedidoKey	TipoPedidoKey	DescuentoImporte	DescuentoImporte
IdControlCarga	IdControlCarga	CostoUnitarioEstimado	CostoUnitarioEstimado

No se mapean: `VentaKey` (IDENTITY), `ImporteBruto`, `VentaNeta`, `CostoTotalEstimado`, `MargenEstimado` (calculadas) y `FechaCargaUTC` (DEFAULT). Las columnas `FechaPedido` y los `…IDOrigen` de dimensiones se quedan en el flujo pero no se guardan: ya cumplieron su función en las búsquedas.

Restricciones validadas por `CHECK_CONSTRAINTS`: `CK_FactVenta_Cantidad` (> 0), `CK_FactVenta_Precio` (> 0), `CK_FactVenta_Costo` (≥ 0), `CK_FactVenta_Descuento` (0 ≤ descuento < cantidad × precio), `CK_FactVenta_Linea` (> 0), las 6 FK a dimensiones y la FK a `ETL_Control`.

Pregunta probable: "¿Por qué FactVenta no tiene un UPDATE como las dimensiones?"
Respuesta: Porque solo se cargan pedidos **completados**, y en Cafeteria un pedido cerrado es **inmutable**: el trigger `TR_Pedido_Integridad` lanza "Un pedido cerrado es histórico e inmutable" y los triggers `TR_DetallePedidoVenta_Abierto` impiden tocar sus líneas. Si el dato no puede cambiar en el origen, el hecho nunca necesita actualizarse: basta con insertar lo nuevo.

Pregunta probable: "¿Qué pasa con un pedido que hoy está abierto y mañana se completa?"
Respuesta: Hoy el `WHERE EstadoPedidoID = 2` lo excluye. Mañana ya cumple el filtro; la búsqueda sobre FactVenta no encuentra sus líneas y las inserta. La carga incremental por clave resuelve los pedidos que se completan tarde sin lógica adicional.

Pregunta probable: "¿Por qué solo la búsqueda de DimTiempo está configurada para fallar?"
Respuesta: Porque una fecha faltante significaría que el calendario quedó mal generado (un error de proceso), y es preferible detener todo y saberlo. Además "Poblar DimTiempo" calcula el rango a partir de las mismas fechas de pedidos, así que en condiciones normales nunca falla.

3.2 FactRecepcionProducto — ETL_FactRecepcionProducto

> **En simple.** Registra **cada producto recibido de un proveedor**: qué llegó, cuánto, a qué costo, a qué sucursal, quién lo recibió y de qué proveedor. Responde "¿cuánto le compramos a cada proveedor por mes?" o "¿cuál es el costo promedio de compra de cada producto?".

Grano: una fila por línea de ingreso (`IngresoID`, `Linea`) — no es una orden de compra, es lo que físicamente se recibió. **Tipo:** transaccional. **Volumen esperado:** ≈ 20 760 filas (432 ingresos semanales simulados × 48 productos + 24 líneas originales).

① Origen_Cafeteria

```
SELECT
    I.IngresoID AS IngresoIDOrigen,
    D.Linea     AS LineaOrigen,

    I.Fecha     AS FechaRecepcion,

    I.SucursalID  AS SucursalIDOrigen,
    I.EmpleadoID  AS EmpleadoIDOrigen,
    D.ProductoID  AS ProductoIDOrigen,
    I.ProveedorID AS ProveedorIDOrigen,

    I.Referencia  AS ReferenciaIngreso,

    D.Cantidad,
    D.CostoUnitario
FROM dbo.IngresoProducto AS I
INNER JOIN dbo.DetalleIngresoProducto AS D
    ON I.IngresoID = D.IngresoID;
```

Parte	Explicación
Cabecera `IngresoProducto` + detalle `DetalleIngresoProducto`	La cabecera dice cuándo, dónde, quién y de qué proveedor; el detalle dice qué producto, cantidad y costo. Mismo patrón cabecera-detalle que la venta.
`I.Fecha AS FechaRecepcion`	Aquí **no hace falta** `CONVERT(date…)`: en el origen `IngresoProducto.Fecha` ya es de tipo `date`.
`I.Referencia AS ReferenciaIngreso`	El número de guía/documento del proveedor. Se guarda como **dimensión degenerada** (texto en el hecho) para poder rastrear el documento físico.
Sin `WHERE`	Los ingresos no tienen estados: todo ingreso registrado es una recepción real.

② Cadena de búsquedas

Orden	Componente	Entra → une con	Devuelve	No encontrado
1	Busqueda_DimTiempo	FechaRecepcion → DimTiempo.Fecha	TiempoKey	Redirige (C)
2	Busqueda_DimSucursal	SucursalIDOrigen → SucursalIDOrigen	SucursalKey	Redirige (C)
3	Busqueda_DimEmpleado	EmpleadoIDOrigen → EmpleadoIDOrigen	EmpleadoKey	Redirige (C)
4	Busqueda_DimProducto	ProductoIDOrigen → ProductoIDOrigen	ProductoKey	Redirige (C)
5	Busqueda_DimProveedor	ProveedorIDOrigen → ProveedorIDOrigen	ProveedorKey	Redirige (C)

Aquí `DimEmpleado` juega el rol de **empleado que recibe** la mercadería (no el que vende), y `DimProducto` se reutiliza: es la misma dimensión conformada que en FactVenta, lo que permite comparar compras y ventas del mismo producto.

③ Agregar_IdControl, ④ Busqueda_FactRecepcion y ⑤ INSERT

```
IdControlCarga = @[User::IdControl]      -- ③ DT_I8

-- ④ une por IngresoIDOrigen + LineaOrigen
SELECT RecepcionKey, IngresoIDOrigen, LineaOrigen FROM dbo.FactRecepcionProducto;
```

⑤ `INSERT_FactRecepcionProducto` mapea 11 columnas: `TiempoKey, SucursalKey, EmpleadoKey, ProductoKey, ProveedorKey, IngresoIDOrigen, LineaOrigen, ReferenciaIngreso, Cantidad, CostoUnitario, IdControlCarga`. No se mapean `RecepcionKey`, `ImporteRecibido` (calculada) ni `FechaCargaUTC`.

Detalle técnico: decimal(22,2). `ImporteRecibido` se declara `decimal(22,2)` porque `Cantidad` es `int` (hasta 10 dígitos) y `CostoUnitario` es `decimal(12,2)`: el producto máximo necesita 22 dígitos. El comentario del script lo dice: "22 dígitos cubren el máximo de int * decimal(12,2) del origen". Así nunca hay desbordamiento aritmético.

Pregunta probable: "¿FactRecepcionProducto es un inventario?"
Respuesta: No. Registra **entradas** (compras recibidas), no el stock. Para inventario haría falta un snapshot periódico de existencias o combinar entradas con salidas (ventas convertidas a insumos), lo cual el origen no modela. El propio script de Cafeteria aclara que "no representa recetas, consumo de insumos ni un libro completo de inventario".

3.3 FactDelivery — ETL_FactDelivery

> **En simple.** Sigue a **cada pedido a domicilio** desde que se pidió, pasando por la salida de la moto, hasta la entrega. Mide cuánto se tardó en preparar, en transportar y en total. Es el único hecho que **se actualiza**: si se cargó cuando el pedido aún no se entregaba, en una ejecución posterior se completa con la hora de entrega.

Grano: una fila por pedido con delivery (`PedidoIDOrigen` único). **Tipo: snapshot acumulativo**. **Volumen esperado:** ≈ 10 616 filas.

¿Por qué un snapshot acumulativo?

Un delivery es un **proceso con hitos**: pedido → salida → entrega. Kimball recomienda una fila por proceso con **una clave de fecha por cada hito** (aquí `TiempoPedidoKey`, `TiempoSalidaKey`, `TiempoEntregaKey`) y medidas de duración entre hitos. La fila se crea con los hitos que ya ocurrieron y se completa después con `UPDATE`. Los comentarios del DM lo declaran: *"DELIVERY: snapshot acumulativo, una fila por PedidoID. Se actualiza al entregar. TiempoEntregaKey NULL significa hito aún no ocurrido"*.

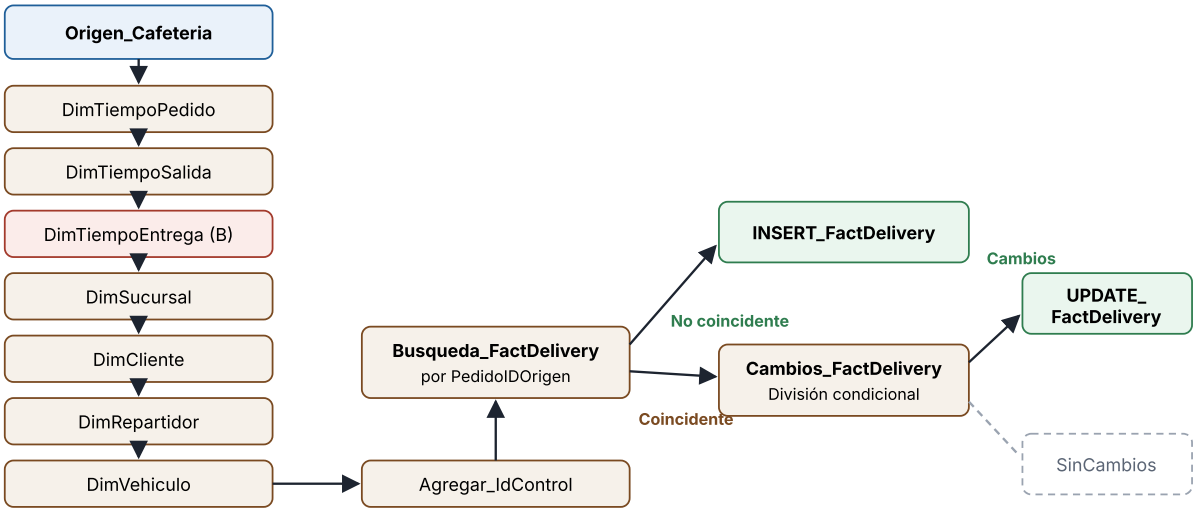


Figura 3. ETL_FactDelivery: 7 búsquedas de dimensión, búsqueda sobre el hecho y rama de actualización.

① Origen_Cafeteria

```
SELECT
    D.PedidoID AS PedidoIDOrigen,

    P.FechaHora AS FechaHoraPedido,
    CONVERT(date, P.FechaHora) AS FechaPedido,

    D.FechaSalida,
    CONVERT(date, D.FechaSalida) AS FechaSalidaDia,

    D.FechaEntrega,
    CONVERT(date, D.FechaEntrega) AS FechaEntregaDia,

    P.SucursalID  AS SucursalIDOrigen,
    P.ClienteID   AS ClienteIDOrigen,

    D.RepartidorID AS RepartidorIDOrigen,
    D.VehiculoID   AS VehiculoIDOrigen,

    D.CargoEnvio
FROM dbo.Delivery AS D
INNER JOIN dbo.PedidoVenta AS P
    ON D.PedidoID = P.PedidoID;
```

Parte	Explicación
`FROM Delivery D INNER JOIN PedidoVenta P`	`Delivery` tiene repartidor, vehículo, salida, entrega y cargo; la hora del pedido, la sucursal y el cliente están en `PedidoVenta`. Delivery es 1:1 con su pedido (su PK es `PedidoID`).
Cada fecha va **dos veces**	Completa (`datetime2`) para guardarla en el hecho y calcular minutos exactos, y como `date` (`…Dia`) para buscarla en DimTiempo.
`CONVERT(date, D.FechaEntrega)`	Si el delivery está pendiente, `FechaEntrega` es NULL y la conversión también da NULL. Es el caso que resuelve la configuración B.
Sin `WHERE EstadoPedidoID = 2`	A propósito: un snapshot acumulativo debe registrar los procesos **en curso** (salió pero no se entregó). Los pedidos cancelados no aparecen porque un trigger del origen impide que un cancelado tenga delivery.
No se extrae `DireccionEntrega`	Es un dato personal de alta cardinalidad que no se usa para agrupar.

② Las 7 búsquedas de dimensión (tres veces DimTiempo)

#	Componente	Entra → une con	Devuelve	No encontrado
1	Busqueda_DimTiempoPedido	FechaPedido → DimTiempo.Fecha	**TiempoPedidoKey**	Redirige (C)
2	Busqueda_DimTiempoSalida	FechaSalidaDia → DimTiempo.Fecha	**TiempoSalidaKey**	Redirige (C)
3	Busqueda_DimTiempoEntrega	FechaEntregaDia → DimTiempo.Fecha	**TiempoEntregaKey**	**Ignora el error (B)** → NULL
4	Busqueda_DimSucursal	SucursalIDOrigen	SucursalKey	Redirige (C)
5	Busqueda_DimCliente	ClienteIDOrigen	ClienteKey	Redirige (C)
6	Busqueda_DimRepartidor	RepartidorIDOrigen → **DimEmpleado**.EmpleadoIDOrigen	**RepartidorKey** (copia de EmpleadoKey)	Redirige (C)
7	Busqueda_DimVehiculo	VehiculoIDOrigen	VehiculoKey	Redirige (C)

> **Dimensiones de rol en acción.** Las búsquedas 1, 2 y 3 consultan la **misma tabla** (`SELECT TiempoKey, Fecha FROM dbo.DimTiempo`) pero cada una devuelve la clave con otro nombre (`TiempoPedidoKey`, `TiempoSalidaKey`, `TiempoEntregaKey`). La búsqueda 6 se llama "DimRepartidor" aunque consulta `DimEmpleado` y renombra `EmpleadoKey` como `RepartidorKey`. En SSIS esto se hace en la pestaña "Columnas" del Lookup, cambiando el *alias de salida*.

③ Agregar_IdControl

Aquí la columna se llama `IdControlEjecucion` (DT_I8, `@[User::IdControl]`) porque cumple **dos papeles**: en el INSERT se guarda en `IdControlCarga` y en el UPDATE en `IdControlActualizacion`.

④ Busqueda_FactDelivery — trae la fila actual completa

```
SELECT DeliveryKey, PedidoIDOrigen,
       TiempoPedidoKey, TiempoSalidaKey, TiempoEntregaKey,
       SucursalKey, ClienteKey, RepartidorKey, VehiculoKey,
       FechaHoraPedido, FechaSalida, FechaEntrega, CargoEnvio
FROM dbo.FactDelivery;
```

Une por `PedidoIDOrigen`. A diferencia de los otros hechos, aquí la salida coincidente **sí está conectada** y devuelve los valores guardados con sufijo `_DM` (`DeliveryKey, TiempoPedidoKey_DM, TiempoSalidaKey_DM, TiempoEntregaKey_DM, SucursalKey_DM, ClienteKey_DM, RepartidorKey_DM, VehiculoKey_DM, FechaHoraPedido_DM, FechaSalida_DM, FechaEntrega_DM, CargoEnvio_DM`) para compararlos. Es el mismo patrón "upsert" de las dimensiones, aplicado a un hecho.

Rama “No coincidente” → INSERT_FactDelivery

Columna del flujo	Columna de FactDelivery	Columna del flujo	Columna de FactDelivery
TiempoPedidoKey	TiempoPedidoKey	PedidoIDOrigen	PedidoIDOrigen
TiempoSalidaKey	TiempoSalidaKey	FechaHoraPedido	FechaHoraPedido
TiempoEntregaKey	TiempoEntregaKey (puede ser NULL)	FechaSalida	FechaSalida
SucursalKey	SucursalKey	FechaEntrega	FechaEntrega (puede ser NULL)
ClienteKey	ClienteKey	CargoEnvio	CargoEnvio
RepartidorKey	RepartidorKey	**IdControlEjecucion**	**IdControlCarga**
VehiculoKey	VehiculoKey	No se mapean: DeliveryKey, Entregada, Minutos..., FechaCargaUTC, IdControlActualizacion, FechaActualizacionUTC (quedan NULL).	

Rama “Coincidente” → Cambios_FactDelivery (División condicional)

```
TiempoPedidoKey != TiempoPedidoKey_DM
|| TiempoSalidaKey != TiempoSalidaKey_DM
|| (ISNULL(TiempoEntregaKey) != ISNULL(TiempoEntregaKey_DM))
|| (!ISNULL(TiempoEntregaKey) && !ISNULL(TiempoEntregaKey_DM) && TiempoEntregaKey != TiempoEntregaKey_DM)
|| SucursalKey != SucursalKey_DM
|| ClienteKey != ClienteKey_DM
|| RepartidorKey != RepartidorKey_DM
|| VehiculoKey != VehiculoKey_DM
|| FechaHoraPedido != FechaHoraPedido_DM
|| FechaSalida != FechaSalida_DM
|| (ISNULL(FechaEntrega) != ISNULL(FechaEntrega_DM))
|| (!ISNULL(FechaEntrega) && !ISNULL(FechaEntrega_DM) && FechaEntrega != FechaEntrega_DM)
|| CargoEnvio != CargoEnvio_DM
```

- Compara **todas** las columnas que el origen puede cambiar mientras el pedido está abierto.
- Las dos columnas que pueden ser NULL (`TiempoEntregaKey` y `FechaEntrega`) usan el mismo truco de la guía de dimensiones: `ISNULL(a) != ISNULL(b)` detecta el paso de “pendiente” a “entregado”, y la segunda condición solo compara cuando ambos tienen valor. Sin esto, comparar NULL haría **fallar** el componente.
- El caso típico que activa la salida **Cambios**: en la carga anterior `FechaEntrega_DM` era NULL y ahora `FechaEntrega` tiene valor.
- **SinCambios** (predeterminada) no está conectada: si nada cambió, no se hace nada.

UPDATE_FactDelivery (Comando de OLE DB)

```
UPDATE dbo.FactDelivery
SET
    TiempoPedidoKey  = ?,   -- Param_0
    TiempoSalidaKey  = ?,   -- Param_1
    TiempoEntregaKey = ?,   -- Param_2
    SucursalKey      = ?,   -- Param_3
    ClienteKey       = ?,   -- Param_4
    RepartidorKey    = ?,   -- Param_5
    VehiculoKey      = ?,   -- Param_6
    FechaHoraPedido  = ?,   -- Param_7
    FechaSalida      = ?,   -- Param_8
    FechaEntrega     = ?,   -- Param_9
    CargoEnvio       = ?,   -- Param_10
    IdControlActualizacion = ?,              -- Param_11 ← IdControlEjecucion
    FechaActualizacionUTC  = SYSUTCDATETIME()
WHERE DeliveryKey = ?;  -- Param_12 ← DeliveryKey (del Lookup)
```

- No se toca `IdControlCarga` ni `FechaCargaUTC`: siguen diciendo **quién creó** la fila. `IdControlActualizacion` y `FechaActualizacionUTC` dicen **quién la modificó por última vez**. La restricción `CK_FactDelivery_Actualizacion` exige que ambas vayan juntas y que la fecha de actualización sea ≥ la de carga.

- `FechaActualizacionUTC` no es parámetro: se calcula en el servidor con `SYSUTCDATETIME()` (hora UTC, independiente de la zona horaria).
- Al actualizar `FechaEntrega`, SQL Server recalcula solas las columnas `Entregada`, `MinutosTransporte` y `MinutosTotales`.
- Se localiza la fila por `DeliveryKey` (PK), que llegó desde la búsqueda.

Ojo con CargoEnvio. El comentario del DM advierte: *"CargoEnvio es cargo registrado, NO ingreso reconocido: no sumar pendientes como ingresos"*. Para reportar ingresos por delivery hay que filtrar `Entregada = 1` o, mejor, usar FactPago (lo efectivamente cobrado).

Pregunta probable: "¿Qué cambiaría si FactDelivery fuera transaccional como los otros?"
Respuesta: Habría que esperar a que el pedido se complete para cargarlo (perdiendo la visibilidad de lo pendiente) o insertar una fila por hito (pedido, salida, entrega), lo que complicaría calcular duraciones. El snapshot acumulativo deja todo el proceso en una sola fila con sus tres fechas y sus minutos.

Pregunta probable: "Con los datos actuales, ¿hay deliveries pendientes?"
Respuesta: No. En la base generada todos los pedidos con delivery están completados y entregados, así que la rama UPDATE recibe 0 filas en la 1.ª carga y la búsqueda de entrega siempre encuentra fecha. Pero el diseño está listo para el caso real en que se ejecute el ETL mientras hay repartos en curso.

3.4 FactPago — ETL_FactPago

> **En simple.** Registra **cada cobro**: cuánto se cobró, con qué método (Efectivo, Visa, Yape…), cuándo, en qué sucursal, a qué cliente y por qué canal. Si un cliente pagó una parte con Yape y otra en efectivo, hay **dos filas**. Responde "¿cuánto se cobra con Yape por mes y por sucursal?".

Grano: una fila por pago (`PagoID`). **No** es un pedido ni una línea de venta (los comentarios del DM lo remarcan). **Tipo:** transaccional. **Volumen esperado:** ≈ 42 376 filas (≈ 20 % de pedidos con pago mixto).

① Origen_Cafeteria

```
SELECT
    PG.PagoID        AS PagoIDOrigen,
    PG.ComprobanteID AS ComprobanteIDOrigen,
    CP.PedidoID      AS PedidoIDOrigen,

    CONVERT(date, PG.FechaHora) AS FechaPago,
    PG.FechaHora AS FechaHoraPago,

    P.SucursalID    AS SucursalIDOrigen,
    P.ClienteID     AS ClienteIDOrigen,
    P.TipoPedidoID  AS TipoPedidoIDOrigen,
    PG.MetodoPagoID AS MetodoPagoIDOrigen,

    CP.Serie        AS SerieComprobante,
    CP.Numero       AS NumeroComprobante,
    CP.FechaEmision AS FechaEmisionComprobante,

    PG.Monto AS MontoCobrado
FROM dbo.Pago AS PG
INNER JOIN dbo.ComprobantePago AS CP ON PG.ComprobanteID = CP.ComprobanteID
INNER JOIN dbo.PedidoVenta     AS P  ON CP.PedidoID    = P.PedidoID
WHERE P.EstadoPedidoID = 2;
```

Parte	Explicación
3 tablas en cadena	`Pago` → `ComprobantePago` → `PedidoVenta`. El pago no conoce la sucursal ni el cliente; los obtiene "subiendo" por el comprobante hasta el pedido.
`WHERE P.EstadoPedidoID = 2`	Mismo criterio que FactVenta: solo cobros de pedidos completados. Así ventas y cobros se pueden **conciliar** (ver 5.2).
`Serie`, `Numero`, `FechaEmision`, `ComprobanteID`, `PedidoID`	Se guardan como **dimensiones degeneradas**: no hay DimComprobante porque cada comprobante es único (no se agrupa por él), pero sirven para rastrear el documento y unir con FactVenta por `PedidoIDOrigen` si se necesita.
`FechaPago` (date) y `FechaHoraPago` (datetime2)	La primera para DimTiempo; la segunda se guarda completa.
Sin cajero	El origen no registra qué empleado cobró, por eso FactPago no tiene `EmpleadoKey` (comentario del DM: "El origen no registra cajero de cobro ni distribución del monto por producto").

② Cadena de búsquedas

Orden	Componente	Entra → une con	Devuelve	No encontrado
1	Busqueda_DimTiempo	FechaPago → DimTiempo.Fecha	TiempoKey	Redirige (C)
2	Busqueda_DimSucursal	SucursalIDOrigen	SucursalKey	Redirige (C)
3	Busqueda_DimCliente	ClienteIDOrigen	ClienteKey	Redirige (C)
4	Busqueda_DimTipoPedido	TipoPedidoIDOrigen	TipoPedidoKey	Redirige (C)
5	Busqueda_DimMetodoPago	MetodoPagoIDOrigen	MetodoPagoKey	Redirige (C)

En este flujo las salidas internas están en español (“Salida de entradas coincidentes de búsqueda”), mientras que en los otros tres hechos están en inglés (“Lookup Match Output”). Es solo la etiqueta según con qué idioma de la extensión se crearon; no cambia el funcionamiento.

③ Agregar_IdControl, ④ Busqueda_FactPago y ⑤ INSERT_FactPago

```
IdControlCarga = @[User::IdControl]                    -- ③
SELECT PagoKey, PagoIDOrigen FROM dbo.FactPago;        -- ④ une por PagoIDOrigen
```

⑤ Mapea 14 columnas: `TiempoKey, SucursalKey, ClienteKey, TipoPedidoKey, MetodoPagoKey, PagoIDOrigen, ComprobanteIDOrigen, PedidoIDOrigen, SerieComprobante, NumeroComprobante, FechaEmisionComprobante, FechaHoraPago, MontoCobrado, IdControlCarga`. Validaciones: `CK_FactPago_Monto` (> 0), `CK_FactPago_Numero` (> 0), `CK_FactPago_Fechas` (pago ≥ emisión del comprobante).

Pregunta probable: “¿Por qué no poner el método de pago en FactVenta?”
Respuesta: Porque tienen **granos distintos**: una venta tiene varias líneas y puede pagarse con varios métodos. Si se pusiera el método en cada línea de venta habría que repartir el monto de forma arbitraria. Por eso se modeló un hecho aparte con su propio grano (el cobro), compartiendo las dimensiones Tiempo, Sucursal, Cliente y TipoPedido: es justamente lo que hace del modelo una **galaxia**.

Pregunta probable: “¿Por qué FactPago tampoco se actualiza?”
Respuesta: El trigger `TR_Pago_Abierto` de Cafeteria impide modificar cobros de pedidos cerrados, y solo se cargan pedidos cerrados (completados). Un pago cargado no puede cambiar en el origen.

Parte 4 · El control de la ejecución

> **En simple.** `ETL_Control` es la **bitácora** del ETL: cada ejecución deja una fila con cuándo empezó, cuándo terminó, si salió bien o mal y cuántas filas de hechos cargó. Los hechos "firman" cada fila con el número de esa ejecución.

4.1 La tabla ETL_Control y la marca de inicio

Columna	Significado
IdControl	`bigint IDENTITY`: número de ejecución. Es lo que reciben los hechos en `IdControlCarga`.
Proceso	Nombre del proceso (`Carga_DM_Cafeteria`).
FechaInicio / FechaFin	Hora UTC de inicio y fin.
UltimaFechaCarga	Marca del **origen** para cargas incrementales por fecha (el paquete la lee pero no la usa para filtrar; ver 5.3).
Estado	`EN_EJECUCION`, `OK` o `ERROR`. El CHECK `CK_ETL_Estado` obliga a que cada estado sea coherente (ej. OK exige FechaFin; ERROR exige mensaje).
FilasProcesadas, NumeroError, Mensaje	Resultado de la ejecución.

El índice único filtrado `UX_ETL_ProcesoActivo ON ETL_Control(Proceso) WHERE Estado='EN_EJECUCION'` garantiza que **no haya dos ejecuciones simultáneas** del mismo proceso.

Tarea "Marca Inicio"

```
exec dbo.SP_ETL_Inicio
@Proceso = ?,              -- Param 0 ← User::Proceso (entrada)
@IdControl = ? output;     -- Param 1 → User::IdControl (salida)
```

Dentro del procedimiento:

```
IF @@TRANCOUNT<>0 OR (2 & @@OPTIONS)=2 THROW 52100, ...;   -- sin transacción externa
SET @Proceso = LTRIM(RTRIM(@Proceso));                       -- limpia espacios
IF @Proceso IS NULL OR LEN(@Proceso)=0 THROW 52101, ...;
BEGIN TRANSACTION;
  SELECT TOP(1) @Anterior = UltimaFechaCarga                  -- marca de la última ejecución OK
  FROM dbo.ETL_Control WITH(UPDLOCK,HOLDLOCK)
  WHERE Proceso=@Proceso AND Estado='OK' ORDER BY IdControl DESC;
  IF EXISTS(... Estado='EN_EJECUCION') THROW 52102, 'Ya existe una ejecucion activa...';
  INSERT dbo.ETL_Control(Proceso, UltimaFechaCarga, Mensaje) VALUES(@Proceso, @Anterior, @Mensaje);
  SET @IdControl = CONVERT(bigint, SCOPE_IDENTITY());          -- el ID recién creado
COMMIT TRANSACTION;
```

- `UPDLOCK, HOLDLOCK`: bloquea el rango leído hasta el COMMIT, para que dos ejecuciones no se "cuelen" al mismo tiempo.
- `SCOPE_IDENTITY()` devuelve el IDENTITY generado en este mismo ámbito (más seguro que `@@IDENTITY`, que podría devolver uno generado por un trigger).
- La fila nace con `Estado = 'EN_EJECUCION'` (DEFAULT) y `FechaInicio = SYSUTCDATETIME()` (DEFAULT).

La tarea siguiente, **Carga Fecha de Inicio**, lee `COALESCE(UltimaFechaCarga, '19000101')` de esa fila y lo deja en `User::FechaInicio`.

4.2 Calcular Filas Procesadas

```
DECLARE @IdControl bigint = ?;          -- ← User::IdControl
SELECT
      (SELECT COUNT_BIG(*) FROM dbo.FactVenta             WHERE IdControlCarga = @IdControl)
    + (SELECT COUNT_BIG(*) FROM dbo.FactRecepcionProducto WHERE IdControlCarga = @IdControl)
    + (SELECT COUNT_BIG(*) FROM dbo.FactDelivery
       WHERE IdControlCarga = @IdControl OR IdControlActualizacion = @IdControl)
    + (SELECT COUNT_BIG(*) FROM dbo.FactPago            WHERE IdControlCarga = @IdControl)
      AS FilasProcesadas;                -- → User::FilasProcesadas (fila única)
```

- Cuenta las filas que **esta** ejecución insertó en cada hecho, gracias a `IdControlCarga`.
- En FactDelivery suma también las filas **actualizadas** en esta ejecución (`IdControlActualizacion`). El `OR` evita contar dos veces una fila que se insertó y actualizó en la misma corrida.
- `COUNT_BIG` devuelve `bigint` (coherente con `FilasProcesadas bigint`).
- Las consultas son rápidas porque existen índices `IX_FactVenta_Control`, `IX_Recepcion_Control`, `IX_FactDelivery_Control`, `IX_FactDelivery_ControlActualizacion` e `IX_FactPago_Control`.

Con los datos generados, la 1.ª ejecución debería dar ≈ 105 905 + 20 760 + 10 616 + 42 376 = **≈ 179 657** filas. Una 2.ª ejecución sin datos nuevos da **0**.

4.3 Fin OK y Fin Error

```
-- Fin OK
EXEC dbo.SP_ETL_Fin_OK @IdControl = ?, @UltimaFechaCarga = NULL, @FilasProcesadas = ?, @Mensaje = ?;

-- dentro del SP: una sola actualización atómica
UPDATE dbo.ETL_Control
SET Estado='OK', FechaFin=SYSUTCDATETIME(),
    UltimaFechaCarga = COALESCE(@UltimaFechaCarga, UltimaFechaCarga),
    FilasProcesadas=@FilasProcesadas, Mensaje=@Mensaje, NumeroError=NULL
WHERE IdControl=@IdControl AND Estado='EN_EJECUCION'
  AND (@UltimaFechaCarga IS NULL OR UltimaFechaCarga IS NULL OR @UltimaFechaCarga>=UltimaFechaCarga);
IF @@ROWCOUNT<>1 THROW 52104, 'Control inexistente, ya finalizado o marca de carga anterior a la vigente.',
1;
```

- Solo puede cerrar una ejecución que esté `EN_EJECUCION` (no se puede cerrar dos veces).
- `@UltimaFechaCarga = NULL` + `COALESCE` → se conserva la marca anterior. La condición final impide que la marca "retroceda" en el tiempo.
- `@@ROWCOUNT<>1`: si no se actualizó exactamente una fila, algo está mal y se lanza error.

```
-- Fin Error (se ejecuta si falla cualquier tarea: restricciones de error con lógica OR)
EXEC dbo.SP_ETL_Fin_Error @IdControl = ?, @Mensaje = ?, @NumeroError = NULL, @FilasProcesadas = ?;
-- dentro: UPDATE ... SET Estado='ERROR', FechaFin=SYSUTCDATETIME(), Mensaje=@Mensaje ...
--         WHERE IdControl=@IdControl AND Estado='EN_EJECUCION'
```

¿Por qué los SP de control exigen "sin transacción externa"? El comentario del DM lo explica: *"Los SP de control se llaman SIN transacción externa para que el log sobreviva al rollback de una futura carga"*. Si el registro de error estuviera dentro de la misma transacción que la carga fallida, al revertirse la carga también se borraría el registro del error.

Algo que conviene saber. El paquete no usa transacciones SSIS (`TransactionOption` por defecto). Si FactPago fallara, las filas que FactVenta, Recepción y Delivery ya insertaron en esa ejecución **se quedan** (cada destino confirma su propio lote). La ejecución queda marcada como ERROR y esas filas se pueden identificar por su `IdControlCarga`. En la siguiente ejecución, las búsquedas sobre cada hecho evitan duplicarlas y solo se carga lo que faltó.

4.4 ¿Qué pasa al ejecutar el paquete otra vez?

Hecho	2.ª ejecución sin datos nuevos	Con datos nuevos en Cafeteria
FactVenta	Todas las líneas salen por "Coincidente" → 0 inserciones	Se insertan solo las líneas de pedidos recién completados
FactRecepcionProducto	0 inserciones	Se insertan las líneas de ingresos nuevos
FactDelivery	Coinciden y no cambian → 0 inserciones, 0 actualizaciones	Deliveries nuevos se insertan; los que se entregaron desde la última carga se actualizan
FactPago	0 inserciones	Se insertan los cobros de pedidos recién completados

Esto es **idempotencia** + **carga incremental por clave natural**. La "red de seguridad" final son las restricciones `UNIQUE` del grano en cada hecho: aunque el flujo estuviera mal, SQL Server rechazaría un duplicado (y el paquete fallaría en vez de duplicar).

Parte 5 · Para la defensa

5.1 Cuadro comparativo de los 4 hechos

Aspecto	FactVenta	FactRecepcionProducto	FactDelivery	FactPago
Grano	Línea de pedido	Línea de ingreso	Pedido con delivery	Pago
Tipo	Transaccional	Transaccional	Snapshot acumulativo	Transaccional
Tablas origen	PedidoVenta, DetallePedidoVenta	IngresoProducto, DetalleIngresoProducto	Delivery, PedidoVenta	Pago, ComprobantePago, PedidoVenta
Filtro	Estado = 2	—	— (incluye pendientes)	Estado = 2
Búsquedas de dimensión	6	5	7 (Tiempo ×3)	5
Búsqueda sobre el hecho	PedidoIDOrigen + LineaOrigen	IngresoIDOrigen + LineaOrigen	PedidoIDOrigen	PagoIDOrigen
Inserta / Actualiza	Sí / No	Sí / No	Sí / **Sí**	Sí / No
Medidas calculadas en el DM	ImporteBruto, VentaNeta, CostoTotal, Margen	ImporteRecibido	Entregada, 3 duraciones	—
Dim. degeneradas	PedidoID, Linea, HoraPedido	IngresoID, Linea, Referencia	PedidoID, fechas completas	PagoID, ComprobanteID, PedidoID, Serie, Número
Filas 1.ª carga (aprox.)	105 905	20 760	10 616	42 376

Los volúmenes se calcularon reproduciendo el generador determinista de `Cafeteria.sql`. Confírmalos con `SELECT COUNT(*)` después de ejecutar el paquete.

5.2 Consultas de conciliación (demostrar que cuadra)

Una forma muy convincente de defender un ETL de hechos es demostrar que **origen y destino cuadran**. Ejecuta estas consultas en SSMS después de cargar:

a) Conteo de filas origen vs. DM

```
SELECT 'Venta' AS Hecho,
  (SELECT COUNT(*) FROM Cafeteria.dbo.PedidoVenta p JOIN Cafeteria.dbo.DetallePedidoVenta d
     ON d.PedidoID=p.PedidoID WHERE p.EstadoPedidoID=2) AS Origen,
  (SELECT COUNT(*) FROM DM_Cafeteria.dbo.FactVenta) AS DM
UNION ALL SELECT 'Recepcion',
  (SELECT COUNT(*) FROM Cafeteria.dbo.DetalleIngresoProducto),
  (SELECT COUNT(*) FROM DM_Cafeteria.dbo.FactRecepcionProducto)
UNION ALL SELECT 'Delivery',
  (SELECT COUNT(*) FROM Cafeteria.dbo.Delivery),
  (SELECT COUNT(*) FROM DM_Cafeteria.dbo.FactDelivery)
UNION ALL SELECT 'Pago',
  (SELECT COUNT(*) FROM Cafeteria.dbo.Pago pg JOIN Cafeteria.dbo.ComprobantePago c ON
c.ComprobanteID=pg.ComprobanteID
     JOIN Cafeteria.dbo.PedidoVenta p ON p.PedidoID=c.PedidoID WHERE p.EstadoPedidoID=2),
  (SELECT COUNT(*) FROM DM_Cafeteria.dbo.FactPago);
-- Las columnas Origen y DM deben coincidir. Si DM fuera menor, alguna búsqueda
-- con salida "No coincidente" desconectada habría descartado filas (ver 5.3).
```

b) Importe de ventas igual a la vista del origen

```
SELECT (SELECT SUM(VentaNeta) FROM Cafeteria.dbo.vw_VentaDetalle) AS Origen,
       (SELECT SUM(VentaNeta) FROM DM_Cafeteria.dbo.FactVenta)  AS DM;
```

c) Lo cobrado = lo vendido + los cargos de envío

```
SELECT
  (SELECT SUM(MontoCobrado) FROM DM_Cafeteria.dbo.FactPago) AS Cobrado,
  (SELECT SUM(VentaNeta) FROM DM_Cafeteria.dbo.FactVenta)
  + (SELECT SUM(CargoEnvio) FROM DM_Cafeteria.dbo.FactDelivery WHERE Entregada = 1) AS VentaMasEnvio;
-- Cafeteria.sql exige en su instalación que ventas completadas + envío = suma de pagos
-- (aserción 51207), por eso en el DM ambos valores deberían coincidir.
```

d) Integridad de la ejecución

```
SELECT IdControl, Estado, FechaInicio, FechaFin, FilasProcesadas, Mensaje
FROM DM_Cafeteria.dbo.ETL_Control ORDER BY IdControl DESC;

SELECT IdControlCarga, COUNT(*) AS Filas
FROM DM_Cafeteria.dbo.FactVenta GROUP BY IdControlCarga;
```

5.3 Riesgos y limitaciones honestas

Observación	Explicación y respuesta
Filas descartadas en silencio	Las búsquedas de dimensión con "redirigir" tienen la salida "No coincidente" sin conectar. Si un ID no existiera en la dimensión, la fila se perdería y el paquete terminaría en OK. Hoy no pasa (las FK del origen y la carga previa de dimensiones lo evitan), y la consulta 5.2-a lo comprueba. **Mejora:** conectar esas salidas a un *Recuento de filas* y fallar si es > 0, o a una tabla de rechazos, o usar un **miembro desconocido** (Key = −1).
INSERT con TABLOCK y UPDATE sobre la misma tabla (FactDelivery)	En el mismo flujo, `INSERT_FactDelivery` usa carga rápida con `TABLOCK` y `UPDATE_FactDelivery` actualiza la misma tabla desde otra conexión. Si en una ejecución hubiera a la vez deliveries nuevos y deliveries actualizados, el UPDATE podría quedar esperando el bloqueo de la carga masiva y el flujo podría quedarse colgado. Con los datos actuales no ocurre (la rama UPDATE recibe 0 filas). **Mejora:** quitar `TABLOCK` en ese destino, o enviar los cambios a una tabla de staging y aplicar un `UPDATE` masivo después del flujo.
Recepciones solo se insertan	A diferencia de pedidos y pagos, `IngresoProducto` no tiene trigger de inmutabilidad: si alguien corrigiera la cantidad de un ingreso ya cargado, FactRecepcionProducto no lo reflejaría. **Respuesta:** en la práctica los ingresos son documentos cerrados; si se quisiera, se aplicaría el mismo patrón de comparación + UPDATE que en FactDelivery.
No se usa la carga incremental por fecha	`User::FechaInicio` se lee pero ningún origen la usa en un `WHERE`, y `Fin OK` envía `@UltimaFechaCarga = NULL`. En cada ejecución se lee **todo** el origen y la búsqueda sobre el hecho descarta lo ya cargado. Con ≈ 180 mil filas es rápido; con millones convendría filtrar por fecha (ej. `WHERE P.FechaCierre > ?`) usando esa variable.
La caché del Lookup sobre el hecho crece	Busqueda_FactVenta carga en memoria todas las filas de FactVenta (3 columnas pequeñas). Con 100 mil filas son pocos MB; con decenas de millones habría que usar caché parcial, un filtro de fecha o comparar en SQL.
UPDATE fila por fila	El Comando OLE DB ejecuta un UPDATE por fila cambiada. Aceptable para pocos deliveries entregados entre ejecuciones; para volúmenes altos, staging + UPDATE masivo.
Sin transacción global	Si falla un hecho, los anteriores quedan cargados (ver 4.3). Es recuperable gracias a `IdControlCarga` y a la idempotencia.
Tipo del parámetro de salida en Marca Inicio	El parámetro `@IdControl` se enlaza con tipo `LONG` (entero de 4 bytes) aunque la variable es Int64. Funciona mientras el número de ejecuciones no supere ≈ 2 100 millones, lo cual no es un problema práctico.

5.4 Banco de preguntas probables del docente

1. ¿Cuál es el propósito de un ETL de hechos?

Cargar los eventos medibles del negocio al grano definido, reemplazando los IDs del origen por las claves sustitutas de las dimensiones, sin duplicar y dejando trazabilidad de qué ejecución cargó cada fila.

2. ¿Qué es el grano de FactVenta y cómo garantizan que no se duplique?

Una línea de pedido completado. Se garantiza con la búsqueda `Busqueda_FactVenta` por (PedidoIDOrigen, LineaOrigen) y, en la base, con `UQ_FactVenta_Origen`.

3. ¿Por qué las búsquedas de dimensión traen solo dos columnas?

Solo se necesita la clave natural para unir y la clave sustituta para devolver; menos columnas = menos memoria en la caché completa.

4. ¿Cómo se busca la fecha en DimTiempo?

El origen convierte `FechaHora` a `date` y el Lookup une esa fecha con `DimTiempo.Fecha` (UNIQUE), devolviendo `TiempoKey`.

5. ¿Qué hace la columna derivada Agregar_IdControl?

Añade a cada fila el número de ejecución (`@[User::IdControl]`, DT_I8) que se guarda en `IdControlCarga`, FK a `ETL_Control`.

6. ¿Qué pasa si una fecha no existe en DimTiempo?

En FactVenta el componente falla (configuración A) y se ejecuta Fin Error. En FactDelivery, para la fecha de entrega, se ignora el error y queda NULL (configuración B). En los demás casos la fila se redirige a una salida no conectada (C).

7. ¿Por qué FactDelivery sí tiene UPDATE?

Porque es un snapshot acumulativo: la fila se crea cuando el delivery salió y se completa cuando se entrega. Los demás hechos cargan eventos de pedidos cerrados, que el origen no permite modificar.

8. ¿Cómo se detecta que un delivery fue entregado desde la última carga?

La búsqueda sobre FactDelivery trae `FechaEntrega_DM` (NULL) y el origen trae `FechaEntrega` con valor; la condición `ISNULL(FechaEntrega) != ISNULL(FechaEntrega_DM)` es verdadera → salida Cambios → UPDATE.

9. ¿Qué es una dimensión de rol? Den un ejemplo del proyecto.

Una dimensión física usada con varios significados: DimTiempo como fecha de pedido, salida y entrega en FactDelivery; DimEmpleado como repartidor (`RepartidorKey`).

10. ¿Qué es una dimensión degenerada?

Un identificador del origen guardado en el hecho sin tabla de dimensión propia: `PedidoIDOrigen`, `LineaOrigen`, `ReferenciaIngreso`, `SerieComprobante`, `NumeroComprobante`.

11. ¿Por qué VentaNeta y MargenEstimado no se calculan en SSIS?

Son columnas calculadas PERSISTED en el DM: la fórmula está en un solo lugar, siempre es consistente y se puede indexar.

12. ¿Por qué FactVenta y FactPago filtran EstadoPedidoID = 2?

Solo los pedidos completados son ventas reales y ya no cambian. Los abiertos se cargarán cuando se completen; los cancelados nunca.

13. ¿Por qué FactDelivery no filtra por estado?

Para poder ver los repartos en curso (snapshot acumulativo). Los cancelados no tienen delivery por regla del origen.

14. ¿Por qué existe FactPago separado de FactVenta?

Tienen granos distintos (cobro vs. línea de venta) y un pedido puede tener pagos mixtos. Comparten dimensiones conformadas: Tiempo, Sucursal, Cliente, TipoPedido.

15. ¿Por qué FactPago no tiene empleado?

El origen no registra quién cobró.

16. ¿Qué cuenta "Calcular Filas Procesadas"?

Las filas de los 4 hechos con `IdControlCarga` igual a la ejecución actual, más las filas de FactDelivery actualizadas en ella (`IdControlActualizacion`).

17. ¿Qué pasa si se ejecuta el paquete dos veces seguidas?

La segunda no inserta ni actualiza nada; FilasProcesadas = 0 y la ejecución termina OK.

18. ¿Qué pasa si falla FactPago?

Se ejecuta Fin Error (estado ERROR). Lo cargado antes en esa ejecución permanece, identificable por su IdControlCarga; la próxima ejecución completa lo faltante sin duplicar.

19. ¿Por qué el UPDATE de FactDelivery no actualiza IdControlCarga?

Para conservar quién creó la fila; quién la modificó queda en `IdControlActualizacion` y `FechaActualizacionUTC`.

20. ¿Por qué se usan fechas en UTC?

Son tiempos técnicos de auditoría; UTC evita ambigüedades por zona horaria o cambios de servidor.

21. ¿Por qué los minutos se calculan con DATEDIFF_BIG en segundos / 60.0?

Para obtener la duración exacta con decimales; DATEDIFF(MINUTE) cuenta cruces de minuto, no duración. Y si la entrega es NULL, el resultado es NULL (no 0).

22. ¿Qué validaciones hace SQL Server al insertar?

Con `CHECK_CONSTRAINTS`: CHECK de cantidades, precios, descuentos, fechas, montos; FK a todas las dimensiones y a ETL_Control; UNIQUE del grano.

23. ¿Qué es la carga incremental en su ETL?

Incremental **por clave**: se lee todo el origen y la búsqueda sobre el hecho deja pasar solo lo que no existe. La infraestructura para incremental por fecha existe (UltimaFechaCarga) pero no se usa.

24. ¿Qué riesgo tienen las salidas "No coincidente" desconectadas?

Que se pierdan filas sin aviso si una clave no existe en la dimensión. Se verifica con la conciliación de conteos y se mejoraría con Recuento de filas, tabla de rechazos o miembro desconocido.

25. ¿Por qué FactVenta guarda HoraPedido y no hay DimHora?

La hora se guarda como atributo del evento (`time(0)`) para análisis por franja horaria sin crear otra dimensión; DimTiempo se mantiene al grano de día.

26. ¿Cómo comprobarían que el ETL de hechos es correcto?

Con conciliaciones: conteos origen vs. DM, SUM(VentaNeta) vs. `vw_VentaDetalle`, y cobrado = vendido + envío; además revisando `ETL_Control`.

27. ¿Por qué SP_ETL_Inicio usa UPDLOCK, HOLDLOCK y un índice único filtrado?

Para impedir dos ejecuciones simultáneas del mismo proceso, incluso si se lanzan al mismo tiempo.

28. ¿Qué mejorarían en los ETL de hechos?

Controlar los "no coincidentes" (Recuento de filas o miembro −1), quitar TABLOCK o usar staging en FactDelivery, filtrar el origen por fecha con UltimaFechaCarga, y aplicar UPDATE en Recepción si los ingresos pudieran corregirse.

5.5 Glosario

Término	Significado
Grano	Lo que representa una fila del hecho.
Medida aditiva / no aditiva	Si se puede sumar por cualquier dimensión (importe) o no (precio unitario).
Hecho transaccional	Una fila por evento; solo inserta.
Snapshot acumulativo	Una fila por proceso con varios hitos; se actualiza al avanzar.
Sustitución de claves	Reemplazar IDs del origen por claves sustitutas mediante búsquedas.
Dimensión degenerada	Identificador guardado en el hecho sin tabla de dimensión.
Dimensión de rol	Una dimensión usada con varios significados en el mismo hecho.
Dimensión conformada	Dimensión compartida por varios hechos.
Columna derivada	Componente SSIS que crea o reemplaza columnas con una expresión.
IgnoreFailure	Disposición de error que deja seguir la fila con NULL en lugar de fallar.
Conciliación	Comparar totales de origen y destino para validar la carga.
Miembro desconocido	Fila especial de una dimensión (ej. Key −1) para hechos cuya clave no se encuentra.
Staging	Tabla intermedia donde se dejan datos para procesarlos en bloque.